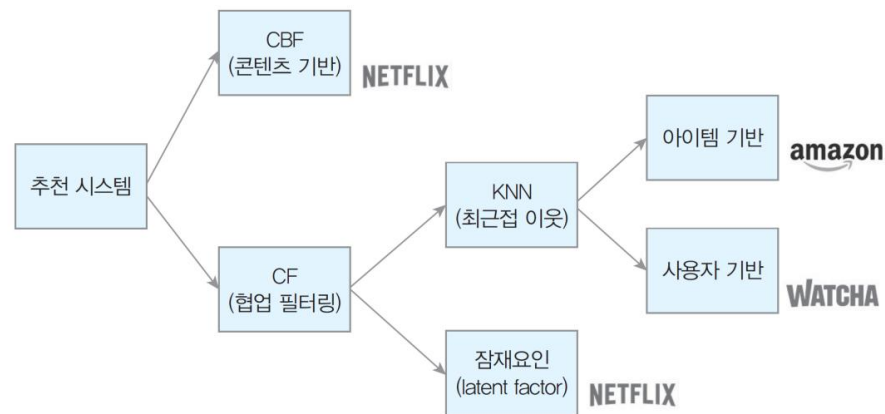


[Chap 15] Recommendation System



recommendation system

- By recommending products and content that match customers' tastes and interests, we encourage customers to experience the service for a longer period and induce additional purchases.
 - Widely used in shopping malls, OTT services, etc., such as Amazon, Coupang, YouTube, and Netflix – Identifying customer preferences based on accumulated data such as purchasing behavior and ratings
- Types of recommendation systems
 - Broadly divided into Content Based Filtering (CBF) and Collaborative Filtering (CF, Classified by Collaborative Filtering)



[그림 15-1] 추천 시스템의 종류

Content Based Filtering (CBF)

- Recommends similar items from the user's previously purchased items based on content
 - Represents content attributes such as movie genre, director, and cast as vectors, and recommends movies with high similarity through similarity evaluation between vectors
 - Advantages include the ability to recommend new or unpopular items without requiring data from other users, and the capacity to explain the reasons for recommendations to users. – Disadvantages include the difficulty of recommending to new customers with insufficient data, the frequent existence of characteristics that are difficult to categorize into tangible values, and the potential for recommendations to become monotonous.

〈표 15-1〉 콘텐츠 기반 필터링

구분	영화제목	장르	감독	출연
내가 이전에 본 영화	Arrival	SF, 드라마, 미스터리	드니 빌뇌브	에이미 아담스 제레미 러너
	프로메테우스	SF, 액션, 스릴러	리들리 스콧	노미 라마스 마이클 패스벤더
	마션	SF, 드라마, 코미디	리들리 스콧	맷 데이먼
▼				
추천 영화	블레이드 러너 2049	SF, 액션, 스릴러	리들리 스콧	라이언 고슬링 해리슨 포드

Collaborative Filtering (CF)

- User-based collaboration filtering

– Find people with similar tastes and recommend items that they rated highly –
 User 3 has similar tendencies to User 1 ∴ User 1 rates Item E highly – What
 item should be recommended to User 3? ∴ E

〈표 15-2〉 사용자 기반 협업 필터링을 위한 평점 행렬

아이템 사용자	아이템 A	아이템 B	아이템 C	아이템 D	아이템 E	아이템 F
사용자 1	4	5	2	1	5	2
사용자 2	1	3	5	4	2	5
사용자 3	5	4	2	2	(데이터 없음)	(데이터 없음)

$$\text{similarity}(1,3) = \frac{(4,5,2,1) \cdot (5,4,2,2)}{|(4,5,2,1)| \cdot |(5,4,2,2)|} = \frac{46}{6.78 \times 7} = 0.97$$

$$\text{similarity}(2,3) = \frac{(1,2,4,4) \cdot (5,4,2,2)}{|(1,2,4,4)| \cdot |(5,4,2,2)|} = \frac{29}{6.08 \times 7} = 0.68$$

Collaborative Filtering (continued)

- Item-based Collaborative Filtering

- Recommends items with similar rating

patterns – User E rates Inception highly → Inception is more similar to Tenet –

What item should be recommended to User E? → Tenet

〈표 15-3〉 아이템 기반 협업 필터링을 위한 평점 행렬

사용자 아이템	사용자 A	사용자 B	사용자 C	사용자 D	사용자 E
인셉션	4	5	3	2	5
테넷	5	5	1	2	(데이터 없음)
미드나잇 인 파리	2	1	4	5	(데이터 없음)

$$\text{similarity}(\text{인셉션}, \text{테넷}) = \frac{(4, 5, 3, 2) \cdot (5, 4, 2, 2)}{|(4, 5, 3, 2)| \cdot |(5, 4, 2, 2)|} = \frac{35}{7.35 \times 7.42} = 0.70$$

$$\text{similarity}(\text{인셉션}, \text{미드나잇 인 파리}) = \frac{(4, 5, 3, 2) \cdot (2, 1, 4, 5)}{|(4, 5, 3, 2)| \cdot |(2, 1, 4, 5)|} = \frac{29}{7.35 \times 6.78} = 0.58$$

Collaborative Filtering (continued)

- Latent factor collaborative filtering

- Using matrix factorization

- Singular Value Decomposition, SVD: $A = U \Sigma V^T$ (here U is $m \times m$, Σ is $m \times n$, V is $n \times n$)

- Truncated SVD

- Since the rating matrix is a very sparse matrix, dimensionality reduction is possible.

$$A \simeq U_k \Sigma_k V_k^T = PQ^T \quad (\text{h: } m \times k), (\text{ : } (k \times n))$$

- The amount of memory decreases from the existing $m \times n$ to $m \times k + k \times n$

Example) If there are 1 million users ($m=1$ million) and 10,000 items ($n=10,000$), the original rating matrix has 10 billion values.

However, if $k=100$, the size of the decomposed matrix decreases significantly to $1 \text{ million} \times 100 + 10,000 \times 100 = 11 \text{ million}$.

- Here, P and Q are not unique

- disadvantage

- Since the exact meaning of latent factors is often unknown, explaining to the user the reason for the recommendation is difficult
- It is difficult to reflect new values when they are added

Collaborative Filtering (continued)

- Example of Latent Factor Collaborative Filtering –
Given User-Item Matrix

〈표 15-4〉 잠재요인 협업 필터링을 위한 평점 행렬

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
사용자 1	4	5	4	1	2
사용자 2		5	5		1
사용자 3	1			5	4
사용자 4	1	1	2	5	

- Decomposition into matrices P and Q

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
사용자 1	4	5	4	1	2
사용자 2		5	5		1
사용자 3	1			5	4
사용자 4	1	1	2	5	

 $=$

	요인 1	요인 2
사용자 1	1.63	-1.49
사용자 2	1.59	-1.77
사용자 3	1.83	1.08
사용자 4	1.93	0.91

 $\cdot$

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
요인 1	1.22	1.32	1.56	1.96	1.73
요인 2	-1.28	-1.74	-1.23	1.31	0.81

[그림 15-2] 특이값 분해 결과

- The product of P and Q approximately restores the original matrix

– Reduce data size –

Provide estimates for missing

values – Recommend to User 2

What about the item? $\ddot{y}$ Alien

	요인 1	요인 2
사용자 1	1.63	-1.49
사용자 2	1.59	-1.77
사용자 3	1.83	1.08
사용자 4	1.93	0.91

 $\cdot$

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
요인 1	1.22	1.32	1.56	1.96	1.73
요인 2	-1.28	-1.74	-1.23	1.31	0.81

 $=$

	에일리언	블레이드러너	프로메테우스	미드나잇 인 파리	라라랜드
사용자 1	3.9	4.8	4.4	1.2	1.6
사용자 2	4.2	5.2	4.7	0.8	1.3
사용자 3	0.9	0.5	1.5	5.0	4.0
사용자 4	1.2	1.0	1.9	5.0	4.1

[그림 15-3] 근사값 복원과 결측치 추정

Implementation of Content-Based Filtering Recommendation System (CBF)

- Utilizing TMDB (The Movie Database) movie data
 - Detailed information on over 4,800 movies, including budget, genre, keywords, language, and title, is organized.

I intend to recommend movies based solely on genre.

〈표 15-5〉 TMDB 데이터셋의 제목과 장르 열

original_title	genres
Avatar	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 878, "name": "Science Fiction"}]
Pirates of the Caribbean: At World's End	[{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 28, "name": "Action"}]
Spectre	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 80, "name": "Crime"}]
The Dark Knight Rises	[{"id": 28, "name": "Action"}, {"id": 80, "name": "Crime"}, {"id": 18, "name": "Drama"}, {"id": 53, "name": "Thriller"}]
John Carter	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 878, "name": "Science Fiction"}]
Spider-Man 3	[{"id": 14, "name": "Fantasy"}, {"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}]
Tangled	[{"id": 16, "name": "Animation"}, {"id": 10751, "name": "Family"}]
Avengers: Age of Ultron	[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 878, "name": "Science Fiction"}]
Harry Potter and the Half-Blood Prince	[{"id": 12, "name": "Adventure"}, {"id": 14, "name": "Fantasy"}, {"id": 10751, "name": "Family"}]

Data Preprocessing

Read the data, extract only the title and genre, and create a separate data frame.

```
1 import pandas as pd
2 movie_full_data = pd.read_csv('tmdb_5000_movies.csv')
3 movie = movie_full_data[['original_title', 'genres']]
```

- Extract only the genre names for each movie and create a list of strings

```
1 from ast import literal_eval
2 for i in range(len(movie)):
3     mv = movie.iloc[i]
4     mv['genres'] = literal_eval(mv['genres'])
5     mv['genres'] = [y['name'] for y in mv['genres']]
6 print(movie)
```

	original_title	genres
0	Avatar	[Action, Adventure, Fantasy, Science Fiction]
1	Pirates of the Caribbean: At World's End	[Adventure, Fantasy, Action]

Data Preprocessing (Continued)

- Convert genres to numeric vectors – Since a

single movie corresponds to multiple genres, create a vector the size of the number of genres and set the value of the corresponding column to 1.

〈표 15-6〉 장르의 벡터화

제목	Action	Adventure	Fantasy	Science Fiction	Crime	Drama	Thriller	Comedy	Romance	...
Avatar	1	1	1	1	0	0	0	0	0	...
Pirates of the Ca..	1	1	1	0	0	0	0	0	0	...
Spectre	1	1	0	0	1	0	0	0	0	...
The Dark Knight..	1	0	0	0	1	1	1	0	0	...
John Carter	1	1	0	1	0	0	0	0	0	...

Data Preprocessing (Continued)

- Convert Genre to Numeric Vector (Continued)
 - List all existing genres: There are a total of 20 genres.

```
1 all_genres = []
2 for i in range(len(movie)):
3     mv = movie.iloc[i]
4     for g in mv['genres']:
5         if g not in all_genres: all_genres.append(g)
6 print(all_genres)
```

```
['Action', 'Adventure', 'Fantasy', 'Science Fiction', 'Crime', 'Drama', 'Thriller',
'Animation', 'Family', 'Western', 'Comedy', 'Romance', 'Horror', 'Mystery', 'History',
'War', 'Music', 'Documentary', 'Foreign', 'TV Movie']
```

- (Line 1) Initialize a list called all_genres – (Line 3) Put the movies from each row into mv
- (Lines 4–5) For each item g of mv['genres'], which is the genre list for each movie
If g is not in all_genres, add

Data Preprocessing (Continued)

- Convert Genres to Numerical Vector (Continued)

- Vectorize the genre for each movie and store it in a gvec column

```

1 import numpy as np
2 for i in range(len(movie)):
3     mv = movie.iloc[i]
4     vec = []
5     for g in all_genres:
6         if g in mv['genres']: vec.append(1)
7         else: vec.append(0)
8     movie.loc[i, 'gvec'] = np.asarray(vec, dtype='float32')
9 print(movie)

```

	original_title	...	gvec
0	Avatar	...	[1.0, 1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...]
1	Pirates of the Caribbean: At World's End	...	[1.0, 1.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...]
2	Spectre	...	[1.0, 1.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, ...]
3	The Dark Knight Rises	...	[1.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0, ...]
4	John Carter	...	[1.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...]
...	...	...	...

- (Line 3) Put each movie data row into mv, and an empty list named vec from line 3 – (Line 5)

Put each genre name from all_genres into g – (Lines

6–7) If g is in the list of movie genre names mv['genres'], append 1 to the vec list; otherwise, append 0.

- (Line 8) Convert the generated vec list into a NumPy array and store it at the corresponding location in the movie dataframe.

Similarity calculation function

- Refer to the cosine similarity function in Chapter 14.

```
1 def magnitude(v):
2     m=0
3     for x in v: m += x**2
4     return (np.sqrt(m))
5
6 def similarity(a,b):
7     dot = np.dot(a,b)
8     if dot==0: return 0
9     ma = magnitude(a)
10    mb = magnitude(b)
11    return(dot/(ma*mb))
```

- (Line 8) Added 'if dot==0: return 0' to the similarity function . • Since there is occasionally data where the genre is not entered, which can cause calculation errors if the absolute value of the denominator becomes 0, return the similarity 0 immediately if the dot product is 0.

User Input and Recommendation Output

- Receives a movie number via keyboard and displays the title of the corresponding movie

```
1 while True:
2     movie_num_str = input("\nEnter movie number: ")
3     if movie_num_str == '': break
4     movie_num = int(movie_num_str)
5     print("You selected: ", movie.loc[movie_num, 'original_title'])
```

– (Line 2) Receive movie number into movie_num_str –

(Line 4) Convert movie number to a numeric value and store it in movie_num – (Line 5) Print the title of the entered movie

- Calculate the similarity value between the input movie and all other movies and add to sim_list

```
1 my_vec = movie.loc[movie_num, 'gvec']
2 sim_list = []
3 for i in range(len(movie)):
4     if i==movie_num: sim_list.append(0)
5     else:
6         mv = movie.iloc[i]
7         sim = similarity(my_vec, mv['gvec'])
8         sim_list.append(sim)
```

User Input and Recommendation Output (Continued)

Find the 10 highest values in the similarity vector and output the titles of the corresponding movies.

```
1 y = np.argsort(sim_list[::-1])
2 print("Recommended:")
3 for i in y[0:10]:
4     print('%4d%i, '%6.4f'%sim_list[i],movie.loc[i,'original_title'])
```

- (Line 1) Create a list of the values at the indices of `sim_list` sorted in descending order and store it in `y`. • If the values of `sim_list` were `[0.8, 0.2, 0.3]`, sorting this in ascending order would result in `[0.2, 0.3, 0.8]`, and `np.argsort` calculates the index order of these values as `[1, 2, 0]`. `[: -1]` :
 - `-1]` reverses the list to make it a descending index.
- (Lines 3–4) Assign the first 10 values of `y` to `i`, and output the corresponding similarity value `sim_list[i]` and the movie title of the corresponding number `movie.loc[i, 'original_title']`.

Full code: ex15-1.py

User Input and Recommendation Output (Continued)

- Output Result –

Confirmed that entering an arbitrary movie number effectively recommends movies of similar genres.

```

Enter movie number: 0
You selected: Avatar
Recommended:
  61 1.0000 Jupiter Ascending
 232 1.0000 The Wolverine
 870 1.0000 Superman II
  10 1.0000 Superman Returns
 813 1.0000 Superman
3494 1.0000 Beastmaster 2: Through the Portal of Time
  14 1.0000 Man of Steel
  46 1.0000 X-Men: Days of Future Past
3208 0.8944 Star Wars: Clone Wars (Volume 1)
1390 0.8944 Underworld: Rise of the Lycans

Enter movie number: 191
You selected: Harry Potter and the Prisoner of Azkaban
Recommended:
 197 1.0000 Harry Potter and the Philosopher's Stone
 341 1.0000 Percy Jackson & the Olympians: The Lightning Thief
 781 1.0000 Inkheart
  37 1.0000 Oz: The Great and Powerful
 143 1.0000 Pan
 114 1.0000 Harry Potter and the Goblet of Fire
 109 1.0000 The Chronicles of Narnia: The Voyage of the Dawn Treader
 368 1.0000 Percy Jackson: Sea of Monsters
 276 1.0000 Harry Potter and the Chamber of Secrets
  63 1.0000 The Chronicles of Narnia: The Lion, the Witch and the Wardrobe

```